###AUTHOR 1: Saad Abdullah, Contribution in EDA & Linear Regression.

- Did the Exploratory Data Analysis.
- Applied Linear Regression with multiple feature selection techniques.

###AUTHOR 2: Abdul Wahab

- Applied Support Vector Regressor (SVR) with RBF kernel and standard scaling
- Performed hyperparameter tuning using GridSearchCV to optimize SVR performance.
- Comparison & Analysis of all 5 models

###AUTHOR 3: Lameya Islam, Contribution on:

- Decision Tree, Random Forest Regressor, Gradient Boosting Regressor(GB) with Outliers
- Decision Tree, Random Forest Regressor, Gradient Boosting Regressor(GB) without Outliers
- Feature Importance in GB

Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Load the Dataset

```
file_url = 'https://raw.githubusercontent.com/wahabmangat/ds-mini-project2/main/boston_housing.csv'
df = pd.read_csv(file_url)

df.head(10)
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 506,\n  \"fields\": [\n    {\n      \"column\": \"Unnamed: 0\",\n      \"properties\": {\n\"dtype\": \"number\",\n        \"std\": 146,\n        \"min\": 0,\n\"max\": 505,\n        \"num_unique_values\": 506,\n\"samples\": [\n          173,\n          274,\n          491\n],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"CRIM\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\":\n8.601545105332487,\n        \"min\": 0.00632,\n        \"max\":\n88.9762,\n        \"num_unique_values\": 504,\n        \"samples\": [\n          0.09178,\n          0.05644,\n          0.10574\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\":\n\"ZN\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
```

```
\"std\": 23.322452994515036,\n        \"min\": 0.0,\n        \"max\":
100.0,\n        \"num_unique_values\": 26,\n        \"samples\": [\n
25.0,\n          30.0,\n          18.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"INDUS\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 6.8603529408975845,\
n        \"min\": 0.46,\n        \"max\": 27.74,\n
\"num_unique_values\": 76,\n        \"samples\": [\n          8.14,\n
1.47,\n          1.22\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"CHAS\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.2539940413404118,\n        \"min\": 0.0,\n        \"max\":
1.0,\n        \"num_unique_values\": 2,\n        \"samples\": [\n
1.0,\n          0.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"NOX\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.11587767566755611,\n        \"min\": 0.385,\n
\"max\": 0.871,\n        \"num_unique_values\": 81,\n
\"samples\": [\n          0.401,\n          0.538\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"RM\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.7026171434153237,\n
\"min\": 3.561,\n        \"max\": 8.78,\n
\"num_unique_values\": 446,\n        \"samples\": [\n          6.849,\
n          4.88\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"AGE\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 28.148861406903638,\n        \"min\": 2.9,\n        \"max\":
100.0,\n        \"num_unique_values\": 356,\n        \"samples\": [\n
51.8,\n          33.8\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"DIS\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 2.1057101266276104,\n        \"min\": 1.1296,\n
\"max\": 12.1265,\n        \"num_unique_values\": 412,\n
\"samples\": [\n          2.2955,\n          4.2515\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"RAD\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 8.707259384239377,\n
\"min\": 1.0,\n        \"max\": 24.0,\n        \"num_unique_values\":
9,\n        \"samples\": [\n          7.0,\n          2.0\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"TAX\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 168.53711605495926,\
n        \"min\": 187.0,\n        \"max\": 711.0,\n
\"num_unique_values\": 66,\n        \"samples\": [\n          370.0,\n
666.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"PTRATIO\",\n      \"properties\": {\n        \"dtype\": \"number\",\
n        \"std\": 2.164945523714446,\n        \"min\": 12.6,\n
```

```
\"max\": 22.0,\n        \"num_unique_values\": 46,\n
\"samples\": [\n          19.6,\n          15.6\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"LSTAT\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 7.141061511348571,\n
\"min\": 1.73,\n        \"max\": 37.97,\n
\"num_unique_values\": 455,\n        \"samples\": [\n          6.15,\n
4.32\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"MEDV\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 9.19710408737982,\n        \"min\": 5.0,\n        \"max\":
50.0,\n        \"num_unique_values\": 229,\n        \"samples\": [\n
14.1,\n          22.5\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```


Data Cleaning

```
# Drop the 'Unnamed: 0' column as it is unnecessary
df = df.drop(columns=['Unnamed: 0'])
df.head(10)
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 506,\n  \"fields\": [\
n    {\n      \"column\": \"CRIM\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 8.601545105332487,\n
\"min\": 0.00632,\n        \"max\": 88.9762,\n
\"num_unique_values\": 504,\n        \"samples\": [\n
0.09178,\n          0.05644,\n          0.10574\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"ZN\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 23.322452994515036,\n
\"min\": 0.0,\n        \"max\": 100.0,\n        \"num_unique_values\":
26,\n        \"samples\": [\n          25.0,\n          30.0,\n
18.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"INDUS\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 6.8603529408975845,\n        \"min\": 0.46,\n        \"max\":
27.74,\n        \"num_unique_values\": 76,\n        \"samples\": [\n
8.14,\n          1.47,\n          1.22\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"CHAS\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.2539940413404118,\n
\"min\": 0.0,\n        \"max\": 1.0,\n        \"num_unique_values\":
2,\n        \"samples\": [\n          1.0,\n          0.0\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"NOX\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\":
0.11587767566755611,\n        \"min\": 0.385,\n        \"max\":
```

```
0.871,\n        \"num_unique_values\": 81,\n        \"samples\": [\n
0.401,\n          0.538\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"RM\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.7026171434153237,\n        \"min\":
3.561,\n        \"max\": 8.78,\n        \"num_unique_values\": 446,\n
\"samples\": [\n          6.849,\n          4.88\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"AGE\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 28.148861406903638,\n
\"min\": 2.9,\n        \"max\": 100.0,\n        \"num_unique_values\":
356,\n        \"samples\": [\n          51.8,\n          33.8\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"DIS\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 2.1057101266276104,\
n        \"min\": 1.1296,\n        \"max\": 12.1265,\n
\"num_unique_values\": 412,\n        \"samples\": [\n
2.2955,\n          4.2515\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"RAD\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 8.707259384239377,\n        \"min\":
1.0,\n        \"max\": 24.0,\n        \"num_unique_values\": 9,\n
\"samples\": [\n          7.0,\n          2.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"TAX\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 168.53711605495926,\n
\"min\": 187.0,\n        \"max\": 711.0,\n
\"num_unique_values\": 66,\n        \"samples\": [\n          370.0,\n
666.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"PTRATIO\",\n      \"properties\": {\n        \"dtype\": \"number\",\
n        \"std\": 2.164945523714446,\n        \"min\": 12.6,\n
\"max\": 22.0,\n        \"num_unique_values\": 46,\n
\"samples\": [\n          19.6,\n          15.6\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"LSTAT\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 7.141061511348571,\n
\"min\": 1.73,\n        \"max\": 37.97,\n
\"num_unique_values\": 455,\n        \"samples\": [\n          6.15,\n
4.32\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"MEDV\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 9.19710408737982,\n        \"min\": 5.0,\n        \"max\":
50.0,\n        \"num_unique_values\": 229,\n        \"samples\": [\n
14.1,\n          22.5\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```

```
# Check for missing values
missing_values = df.isnull().sum()
missing_values
```

```
CRIM       0
ZN         0
INDUS      0
CHAS       0
NOX        0
RM         0
AGE        0
DIS        0
RAD        0
TAX        0
PTRATIO    0
LSTAT      0
MEDV       0
dtype: int64
```

```
#summary_statistics
df.describe()
```

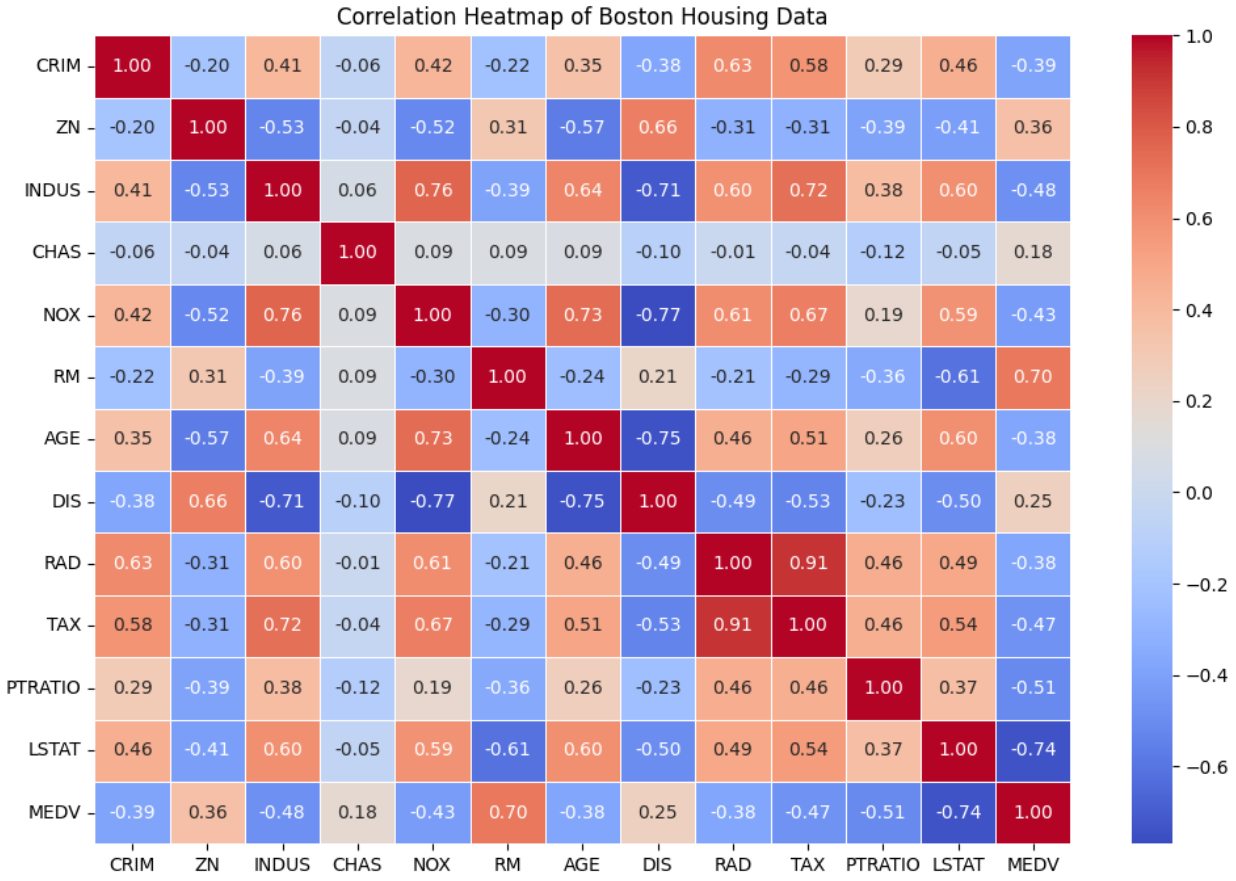

```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": \"CRIM\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 176.21241273856964,\n
\"min\": 0.00632,\n        \"max\": 506.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
3.613523557312254,\n          0.25651,\n          506.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"ZN\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 174.65631992520625,\n
\"min\": 0.0,\n        \"max\": 506.0,\n        \"num_unique_values\":
6,\n        \"samples\": [\n          506.0,\n
11.363636363636363,\n          100.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"INDUS\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 175.10046881853455,\
n        \"min\": 0.46,\n        \"max\": 506.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
11.13677865612648,\n          9.69,\n          506.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"CHAS\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 178.83151296515905,\n
\"min\": 0.0,\n        \"max\": 506.0,\n        \"num_unique_values\":
5,\n        \"samples\": [\n          0.0691699604743083,\n
1.0,\n          0.2539940413404118\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"NOX\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 178.71946937975397,\n
\"min\": 0.1158776756675561 1,\n        \"max\": 506.0,\n
```

```
\"num_unique_values\": 8,\n        \"samples\": [\n
0.5546950592885376,\n          0.538,\n          506.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"RM\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 176.99257138815915,\n
\"min\": 0.7026171434153237,\n        \"max\": 506.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
6.284634387351779,\n          6.2085,\n          506.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"AGE\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 161.29423343904304,\n
\"min\": 2.9,\n        \"max\": 506.0,\n        \"num_unique_values\":
8,\n        \"samples\": [\n          68.57490118577076,\n
77.5,\n          506.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"DIS\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 177.4338019618181,\n        \"min\": 1.1296,\n
\"max\": 506.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          3.795042687747036,\n
3.2074499999999997,\n          506.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"RAD\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 175.26272292595038,\n
\"min\": 1.0,\n        \"max\": 506.0,\n        \"num_unique_values\":
7,\n        \"samples\": [\n          506.0,\n
9.549407114624506,\n          5.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"TAX\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 205.93933614417855,\n
\"min\": 168.53711605495926,\n        \"max\": 711.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
408.2371541501976,\n          330.0,\n          506.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"PTRATIO\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
173.36059244426343,\n        \"min\": 2.164945523714446,\n
\"max\": 506.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          18.455533596837945,\n          19.05,\n
506.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"LSTAT\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 174.45535325169888,\n        \"min\": 1.73,\n        \"max\":
506.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
12.653063241106722,\n          11.36,\n          506.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"MEDV\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 171.852511161592,\n
\"min\": 5.0,\n        \"max\": 506.0,\n        \"num_unique_values\":
8,\n        \"samples\": [\n          22.532806324110677,\n
```

```
21.2,\n          506.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```

Heatmap for Correlation

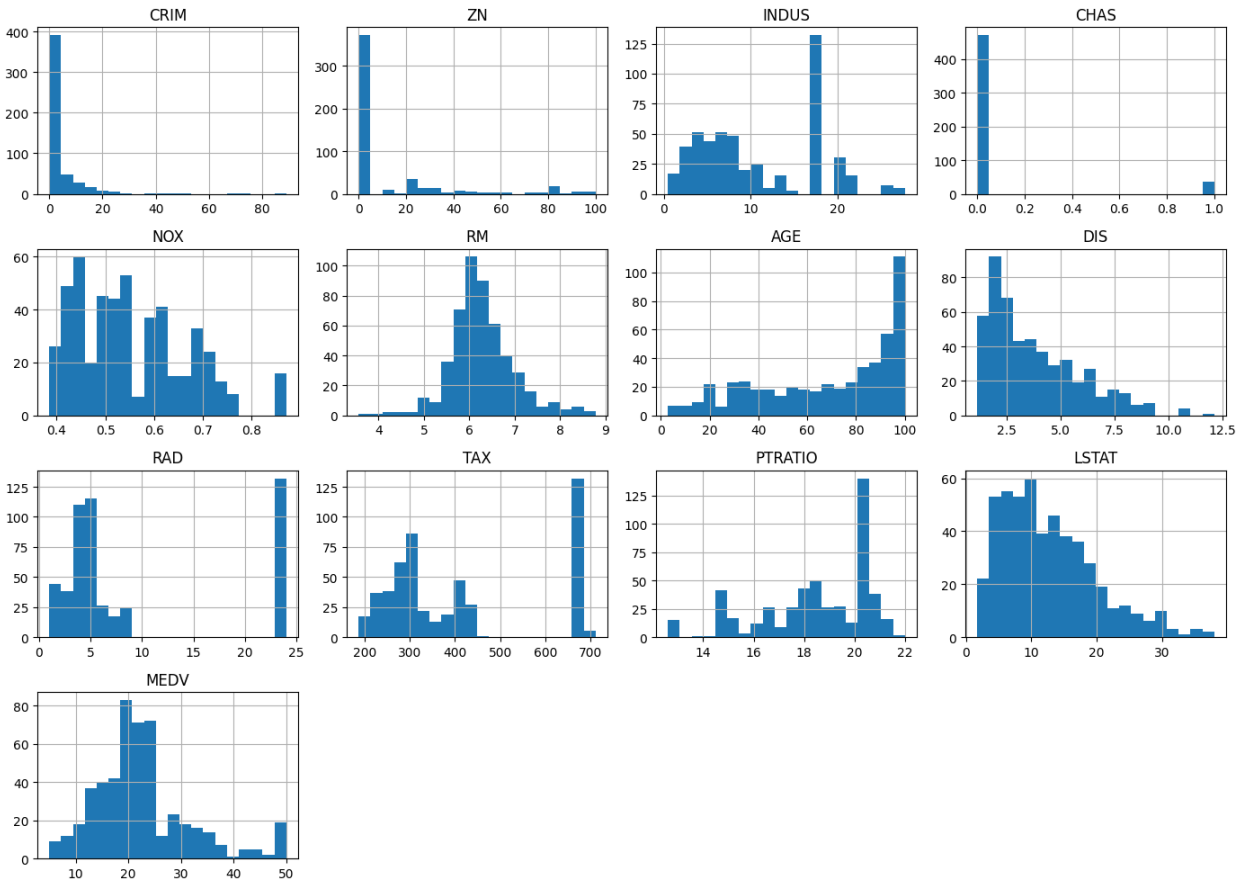
```
plt.figure(figsize=(12, 8))
correlation_matrix = df.corr()

sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm",
fmt='.2f', linewidths=0.5)
plt.title('Correlation Heatmap of Boston Housing Data')
plt.show()
```



#Histogram

For Viewing the distribution of features

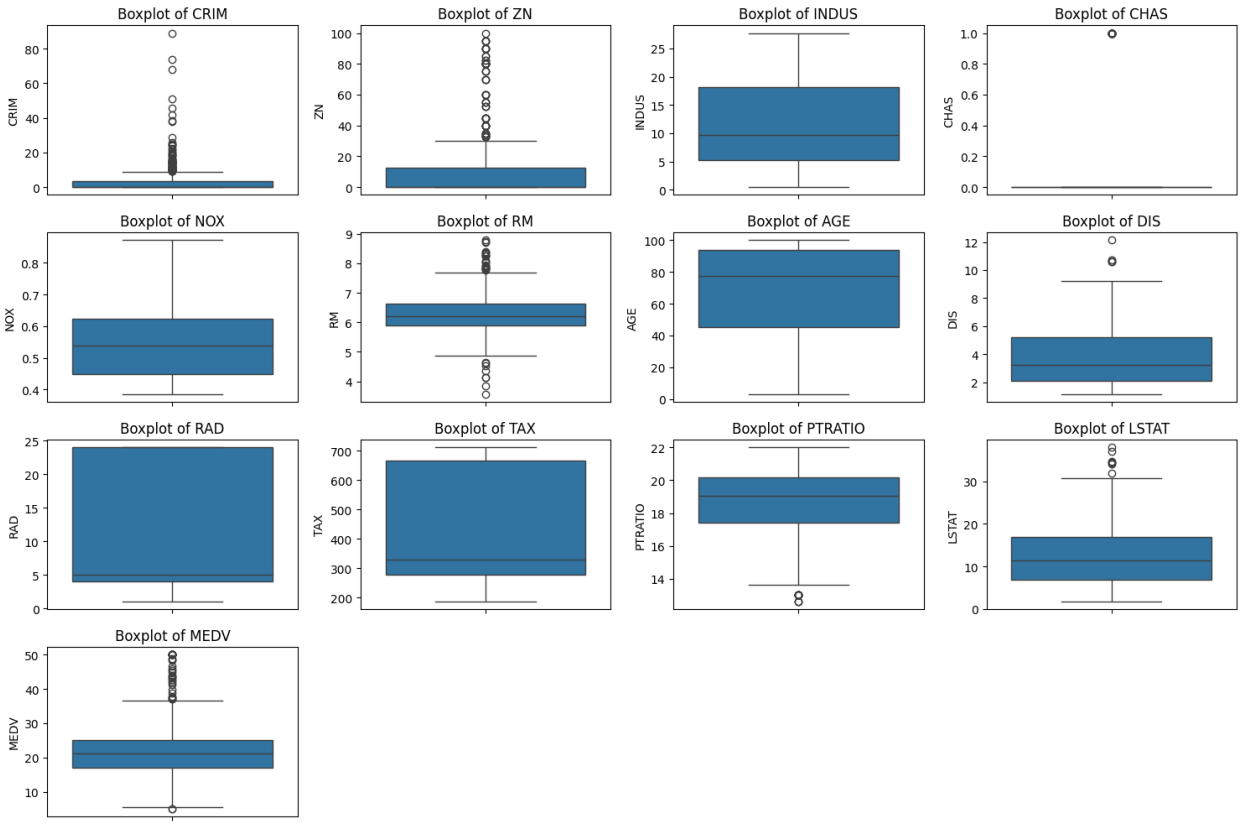
```
df.hist(bins=20, figsize=(14, 10), layout=(4, 4))
plt.tight_layout()
plt.show()
```



Box plots

to see if there are outliers in data.

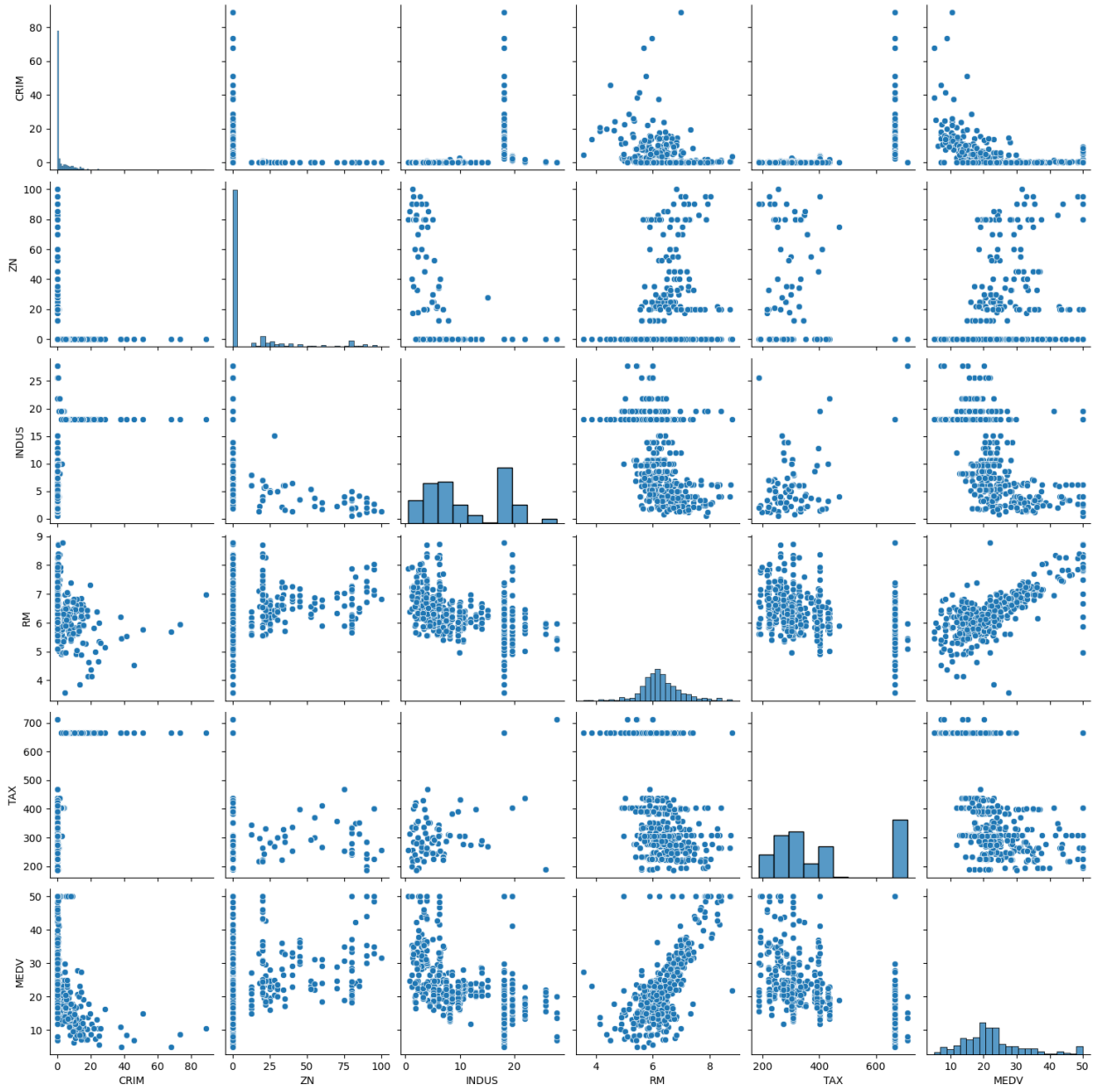
```
plt.figure(figsize=(15, 10))

for i, column in enumerate(df.columns, 1):
    plt.subplot(4, 4, i)  # Adjust subplot grid size based on the
number of features
    sns.boxplot(y=df[column])
    plt.title(f'Boxplot of {column}')

plt.tight_layout()
plt.show()
```



Pair plots

to see more precisely the relationship between different attributes. (these attributes are chosen after observing Heatmap)

```
sns.pairplot(df[['CRIM', 'ZN', 'INDUS', 'RM', 'TAX', 'MEDV']])
plt.show()
```



Linear Regression

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Here's a summary of the approaches I have tried with linear regression and the final results:

1. All Features Without Scaling: Applied linear regression on all features without scaling.
2. All Features With Scaling: Applied linear regression on all features with feature scaling.

3. Limited Features (Recursive Feature Elimination): Tried using a subset of features.
4. Lasso Regression: Applied Lasso regression for feature selection and regularization.
5. Filter data for outliers using z-score and then simple linear regression.

Best Model:

- Approach: Linear regression using all features without feature scaling (normalization).
- Test R-squared: 0.7166209449097798
- Mean Squared Error: 21.115450613869438

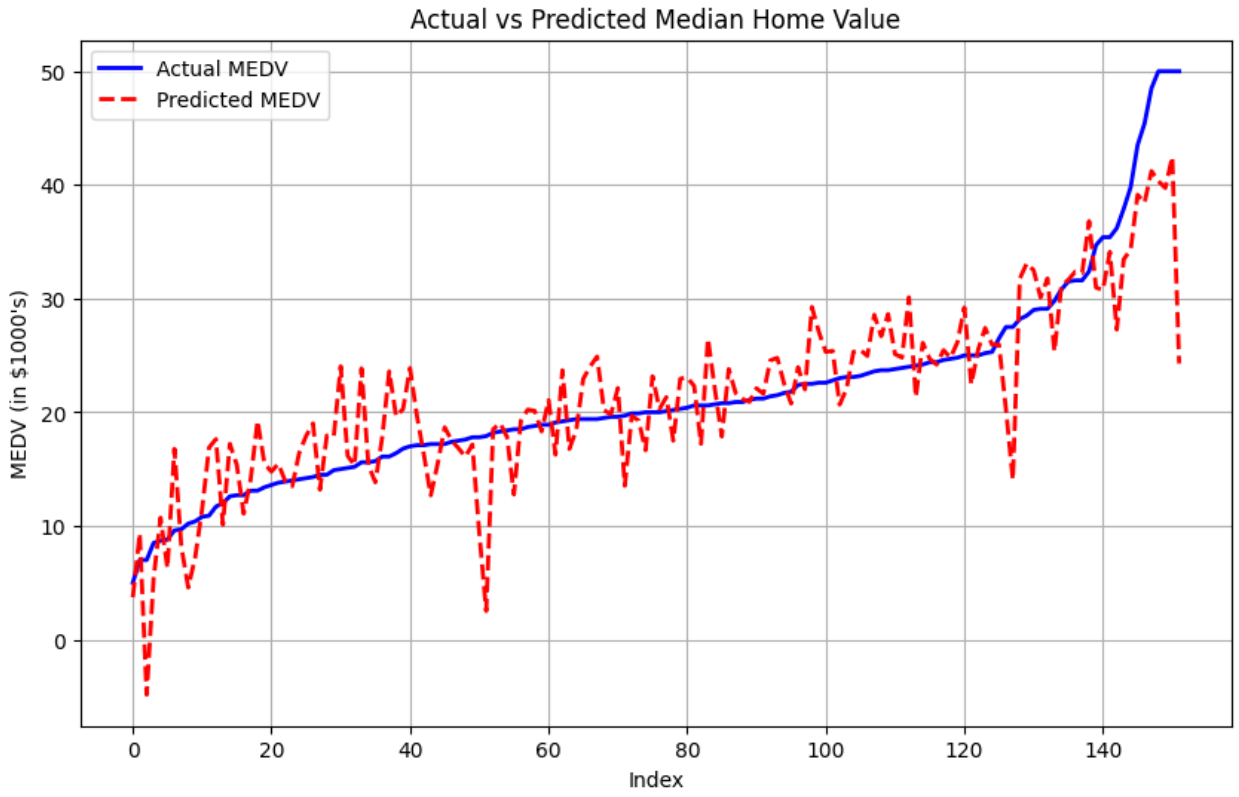
Conclusion: I have tried multiple things with linear regression, even with different train test splits, but so far, for me, using all features without feature scaling and 70/30 train test split was on top with performance.

PS: For model evaluation I have used following 2 matrices:

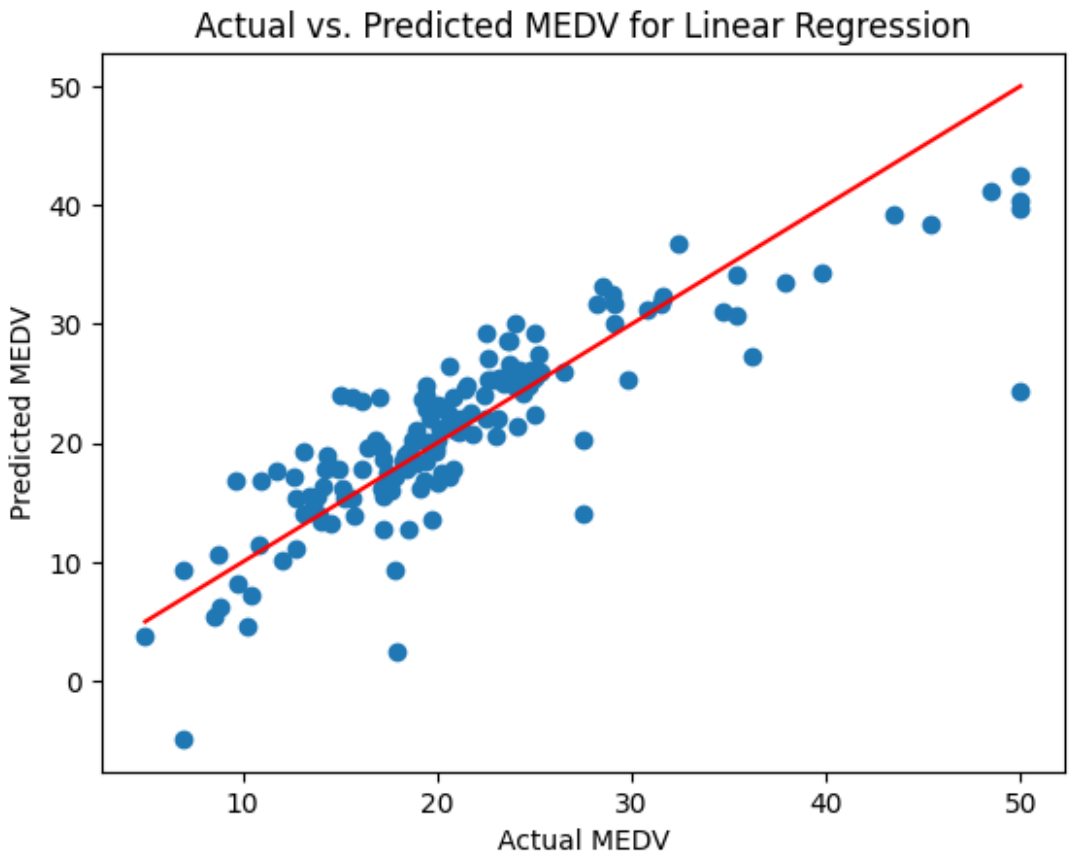
- Mean Squared Error (MSE): Measures the average of the squared differences between the actual and predicted values. Lower values indicate better model performance.
- R-squared: Indicates how well the model explains the variability of the target variable. Values closer to 1 are better.
- I have only added best performing LR model here

```
# using all features other than MEDV which obviously is the target.
X = df.drop(columns=['MEDV'])
y = df['MEDV']  # Target variable

# Split data (70% training, 30% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Linear Regression (without feature scaling)
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
y_pred_test = lr_model.predict(X_test)

# Model evaluation
test_mse = mean_squared_error(y_test, y_pred_test)
test_r2 = r2_score(y_test, y_pred_test)

print(f'Mean Squared Error: {test_mse}')
print(f'R-squared: {test_r2}')

Mean Squared Error: 21.115450613869438
R-squared: 0.7166209449097798

# Sort the test set for a cleaner line chart
sorted_index = np.argsort(y_test)
y_test_sorted = y_test.iloc[sorted_index]
y_pred_sorted = y_pred_test[sorted_index]

# Plot actual vs predicted values as line charts
```

```
plt.figure(figsize=(10, 6))
plt.plot(y_test_sorted.values, label='Actual MEDV', color='b',
linewidth=2)
plt.plot(y_pred_sorted, label='Predicted MEDV', color='r',
linestyle='--', linewidth=2)

# Add labels, title, and legend
plt.xlabel("Index")
plt.ylabel("MEDV (in $1000's)")
plt.title("Actual vs Predicted Median Home Value")
plt.legend()
plt.grid(True)
plt.show()
```




```
plt.scatter(y_test, y_pred_test)
plt.xlabel("Actual MEDV")
plt.ylabel("Predicted MEDV")
plt.title("Actual vs. Predicted MEDV for Linear Regression")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red')  # Line of best fit
plt.show()
```



Decision Tree, Random Forest Regressor, Gradient Boosting Regressor with Outliers

Train-Test split has been done on (70-30)% of data.

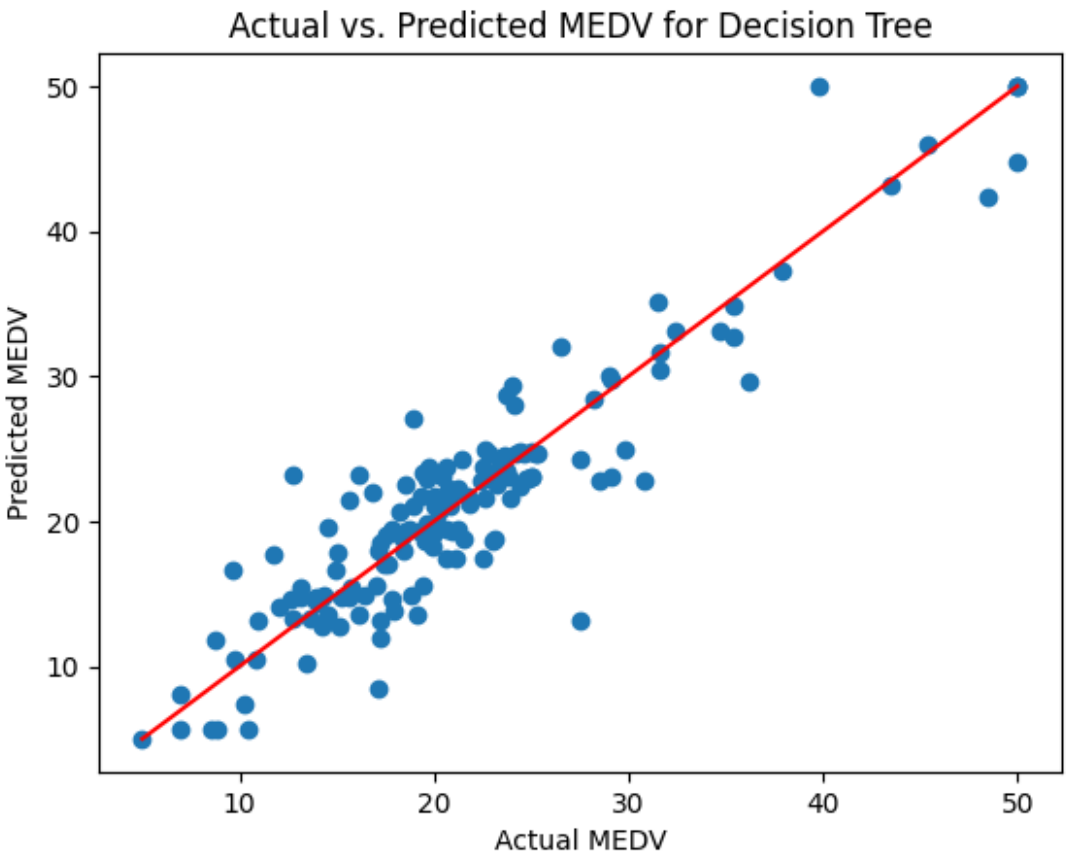
```
# Import necessary libraries
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Dictionary to store models and their results
model_results = {}

# Step 1: Decision Tree Regressor
dt_model = DecisionTreeRegressor(random_state=42)
dt_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_dt = dt_model.predict(X_test)
```

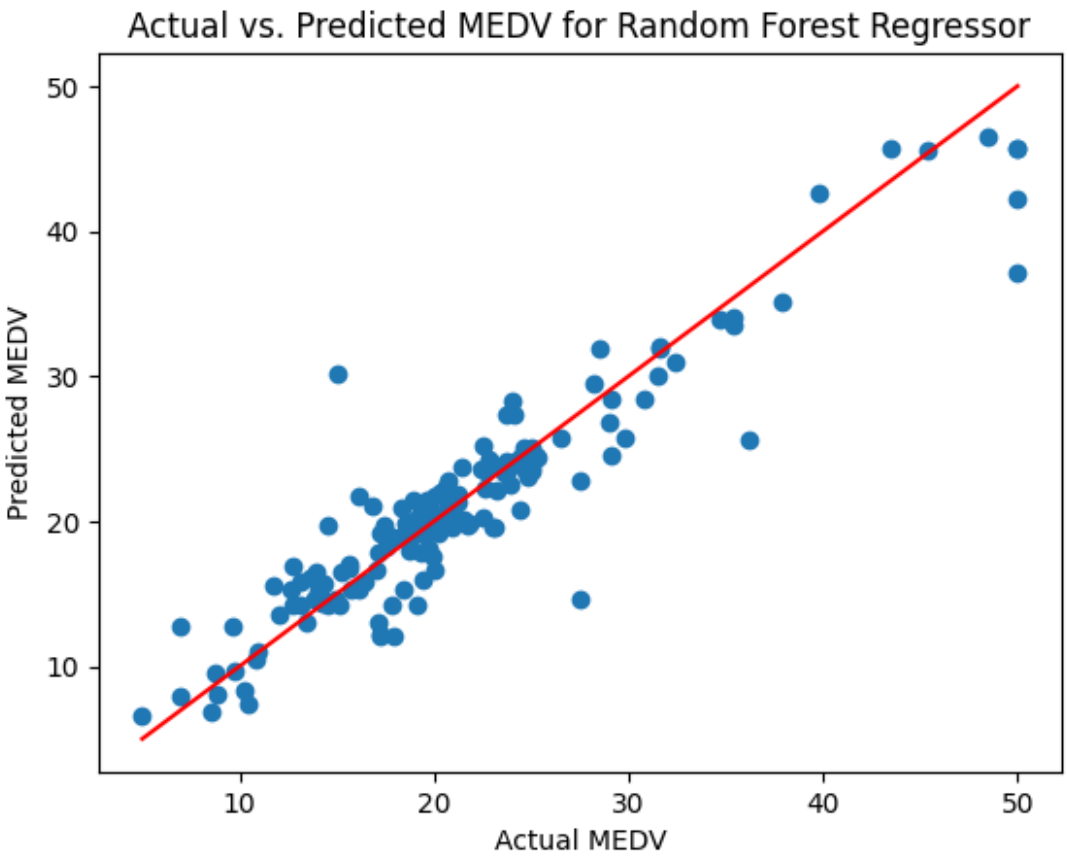
```python
dt_mse = mean_squared_error(y_test, y_pred_dt)
dt_r2 = r2_score(y_test, y_pred_dt)

# Store results
model_results['Decision Tree'] = {'MSE': dt_mse, 'R2': dt_r2}

# Step 2: Random Forest Regressor
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_rf = rf_model.predict(X_test)
rf_mse = mean_squared_error(y_test, y_pred_rf)
rf_r2 = r2_score(y_test, y_pred_rf)

# Store results
model_results['Random Forest'] = {'MSE': rf_mse, 'R2': rf_r2}

# Step 3: Gradient Boosting Regressor
gb_model = GradientBoostingRegressor(n_estimators=100,
learning_rate=0.1, random_state=42)
gb_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_gb = gb_model.predict(X_test)
gb_mse = mean_squared_error(y_test, y_pred_gb)
gb_r2 = r2_score(y_test, y_pred_gb)

# Store results
model_results['Gradient Boosting'] = {'MSE': gb_mse, 'R2': gb_r2}

# Print all results
for model_name, metrics in model_results.items():
    print(f"\n{model_name} Results:")
    print(f"Test Mean Squared Error: {metrics['MSE']}")
    print(f"Test R-squared: {metrics['R2']}")
```

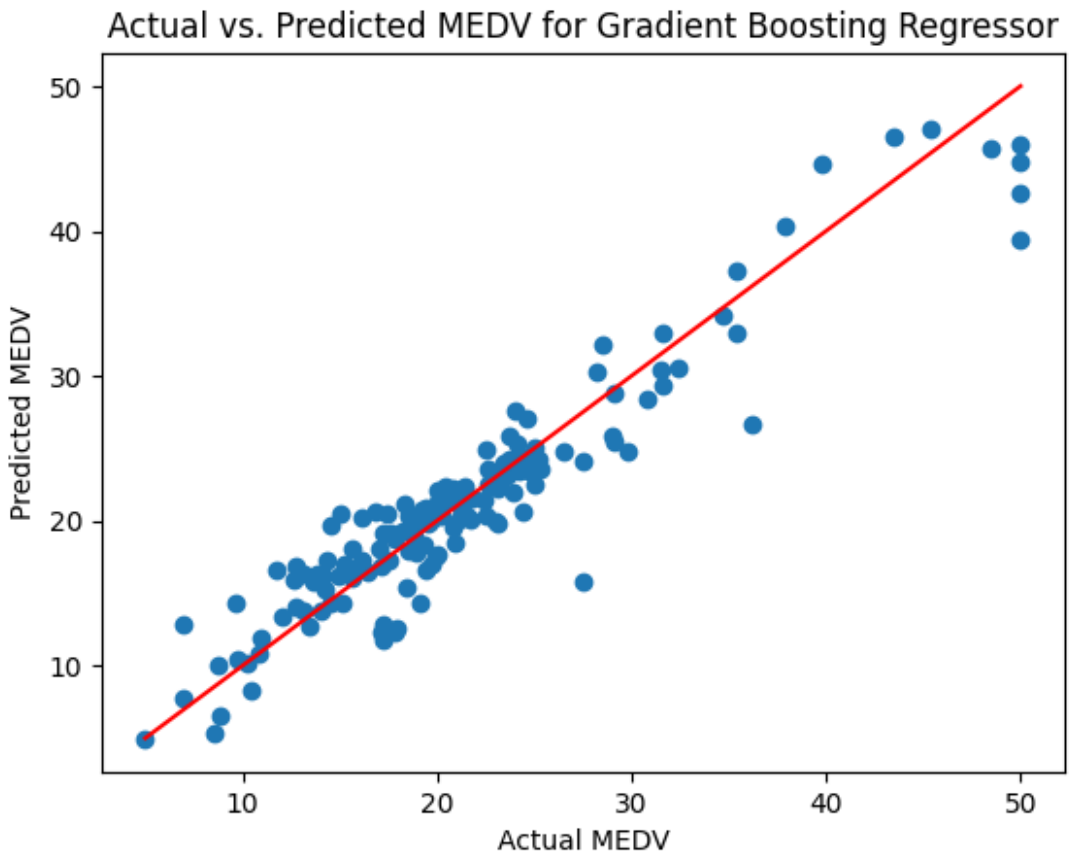
```
Decision Tree Results:
Test Mean Squared Error: 11.282500000000002
Test R-squared: 0.8485836628579762

Random Forest Results:
Test Mean Squared Error: 9.686613907894738
Test R-squared: 0.8700011879244487

Gradient Boosting Results:
Test Mean Squared Error: 8.126460502912597
Test R-squared: 0.8909391639015861
```

Among these models, Gradient Boosting Regressor(GB) has better performance as the R-squared value is higher than others. And GB performs better because of its sequential error correction, flexibility in handling complex, non-linear relationships, and robustness against overfitting through regularization. Additionally, its ability to handle different data types and integrate regularization techniques (like learning rate and sub-sampling) further boosts its performance

```
plt.scatter(y_test, y_pred_dt)
plt.xlabel("Actual MEDV")
plt.ylabel("Predicted MEDV")
plt.title("Actual vs. Predicted MEDV for Decision Tree")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red')  # Line of best fit
plt.show()

plt.scatter(y_test, y_pred_rf)
plt.xlabel("Actual MEDV")
plt.ylabel("Predicted MEDV")
plt.title("Actual vs. Predicted MEDV for Random Forest Regressor")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red')  # Line of best fit
plt.show()

plt.scatter(y_test, y_pred_gb)
plt.xlabel("Actual MEDV")
plt.ylabel("Predicted MEDV")
plt.title("Actual vs. Predicted MEDV for Gradient Boosting Regressor")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red')  # Line of best fit
plt.show()
```

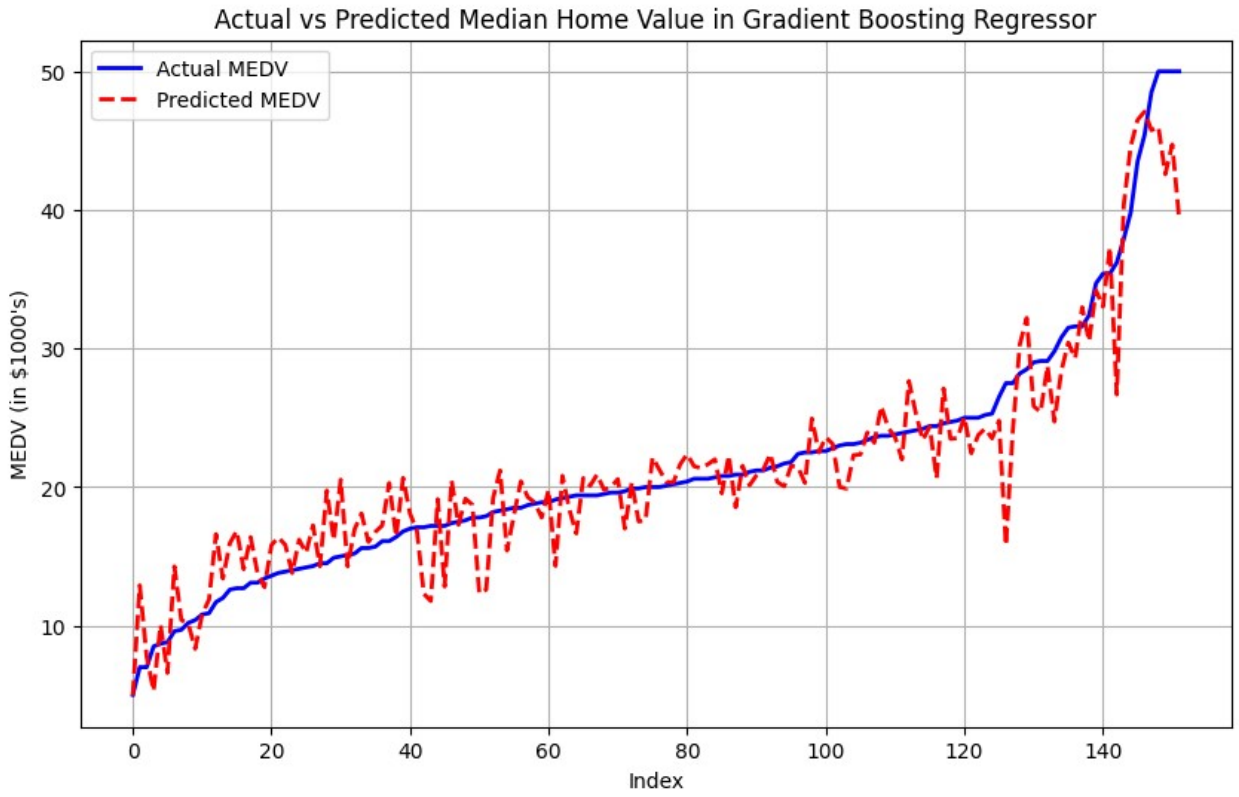

Actual vs. Predicted MEDV for Decision Tree
Predicted MEDV
Actual MEDV
10
20
30
40
50


Actual vs. Predicted MEDV for Random Forest Regressor
Predicted MEDV
Actual MEDV
10
20
30
40
50



From the visualization, some outliers can be noticed above the value of 40 in MEDV.

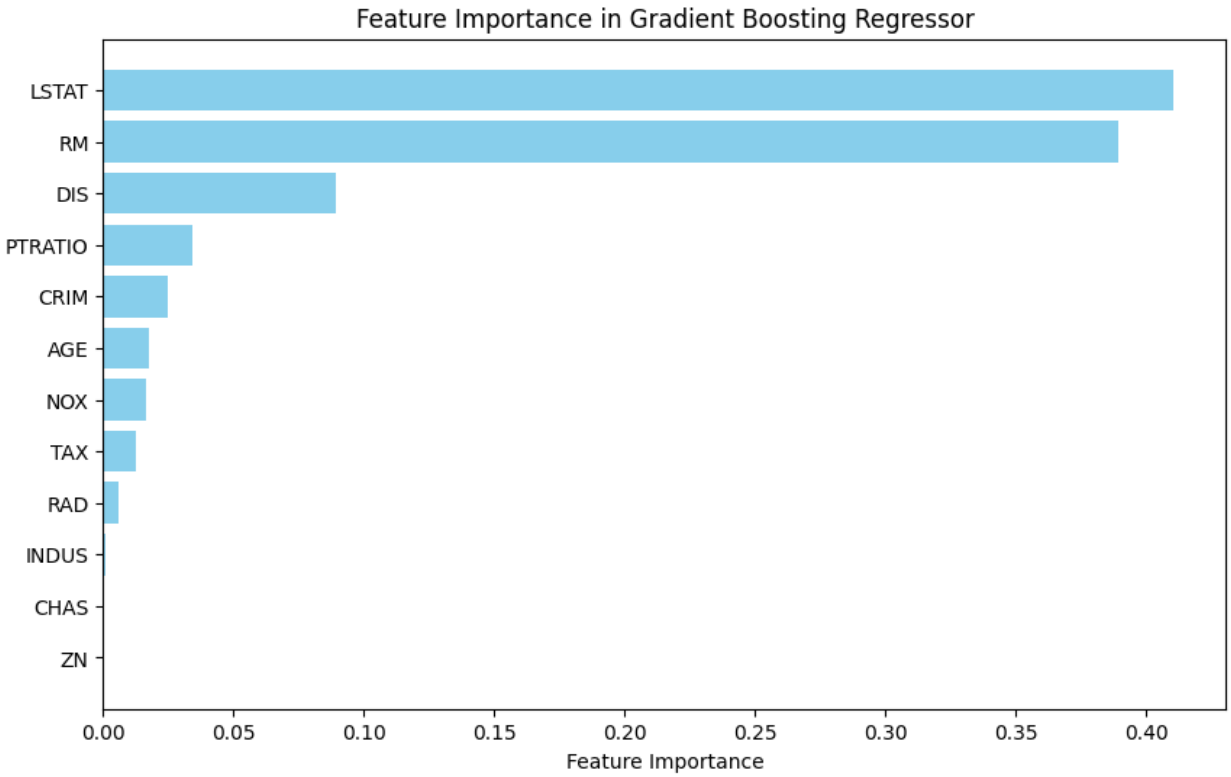
```
# Sort the test set for a cleaner line chart
sorted_index = np.argsort(y_test)
y_test_sorted = y_test.iloc[sorted_index]
y_pred_sorted = y_pred_gb[sorted_index]

# Plot actual vs predicted values as line charts
plt.figure(figsize=(10, 6))
plt.plot(y_test_sorted.values, label='Actual MEDV', color='b',
linewidth=2)
plt.plot(y_pred_sorted, label='Predicted MEDV', color='r',
linestyle='--', linewidth=2)

# Add labels, title, and legend
plt.xlabel("Index")
plt.ylabel("MEDV (in $1000's)")
plt.title("Actual vs Predicted Median Home Value in Gradient Boosting
Regressor")
plt.legend()
plt.grid(True)
plt.show()
```



Important Feature Selection

As, Gradient Boosting Regressor(GB) performs better. So, I tried to observe the important feature for GB.

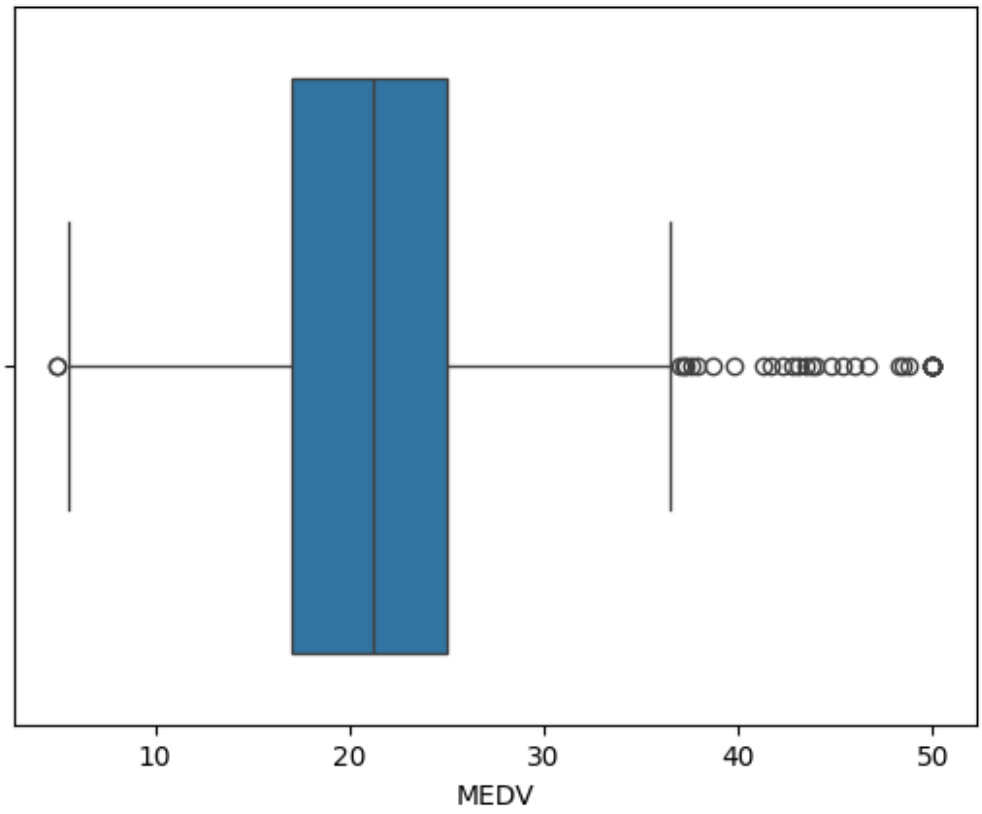
```
model = gb_model
importance_df = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': model.feature_importances_
})

importance_df = importance_df.sort_values(by='Importance',
ascending=False)
importance_df = importance_df[importance_df['Importance'] > 0]

plt.figure(figsize=(10, 6))
plt.barh(importance_df['Feature'], importance_df['Importance'],
color='skyblue')
plt.xlabel('Feature Importance')
plt.title('Feature Importance in Gradient Boosting Regressor')
plt.gca().invert_yaxis()
plt.show()
```



Decision Tree, Random Forest Regressor, Gradient Boosting Regressor without Outliers

```
#Checking outliers through boxplot
import seaborn as sns
sns.boxplot(x=df['MEDV'])
```

```
<Axes: xlabel='MEDV'>
```



The circles plotted outside the whiskers on the right side (above the value of 40 for MEDV) are outliers. These points are much higher than the majority of the data.

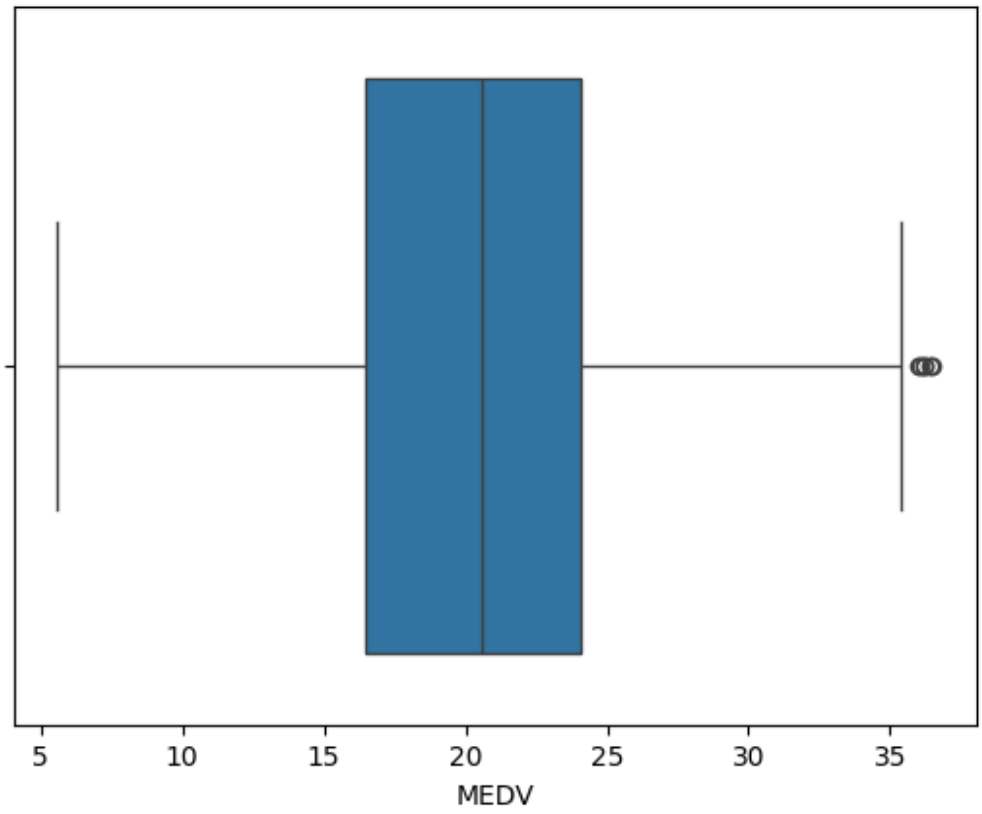
The boxplot suggests that the interquartile range (IQR), which is the distance between the first quartile (Q1) and the third quartile (Q3), defines the central range of the data. Data points beyond 1.5 times the IQR from Q1 and Q3 are considered outliers.

Interquartile Range (IQR):

The IQR method defines outliers as values lying outside 1.5 times the IQR from the lower (25th) or upper (75th) quartile

```
Q1 = df['MEDV'].quantile(0.25)
Q3 = df['MEDV'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df_clean = df[(df['MEDV'] >= lower_bound) & (df['MEDV'] <=
upper_bound)]

import seaborn as sns
sns.boxplot(x=df_clean['MEDV'])
```

```
<Axes: xlabel='MEDV'>
```




```
# using all features other than MEDV which obviously is the target.
X = df_clean.drop(columns=['MEDV'])
y = df_clean['MEDV']  # Target variable

# Split data (70% training, 30% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Dictionary to store models and their results
model_results = {}

# Step 1: Decision Tree Regressor
dt_model = DecisionTreeRegressor(random_state=42)
dt_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_dt = dt_model.predict(X_test)
dt_mse = mean_squared_error(y_test, y_pred_dt)
dt_r2 = r2_score(y_test, y_pred_dt)

# Store results
model_results['Decision Tree'] = {'MSE': dt_mse, 'R2': dt_r2}
```

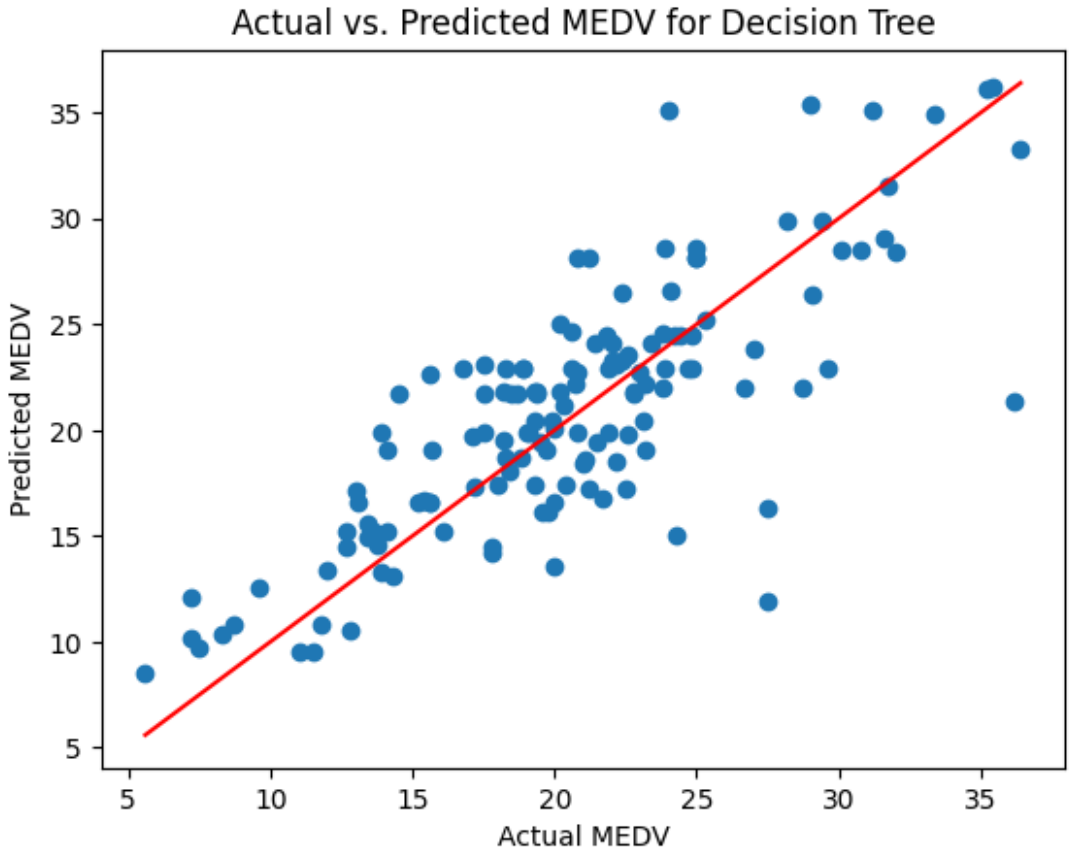
```
# Step 2: Random Forest Regressor
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_rf = rf_model.predict(X_test)
rf_mse = mean_squared_error(y_test, y_pred_rf)
rf_r2 = r2_score(y_test, y_pred_rf)

# Store results
model_results['Random Forest'] = {'MSE': rf_mse, 'R2': rf_r2}

# Step 3: Gradient Boosting Regressor
gb_model = GradientBoostingRegressor(n_estimators=100,
learning_rate=0.1, random_state=42)
gb_model.fit(X_train, y_train)

# Predict and evaluate
y_pred_gb = gb_model.predict(X_test)
gb_mse = mean_squared_error(y_test, y_pred_gb)
gb_r2 = r2_score(y_test, y_pred_gb)

# Store results
model_results['Gradient Boosting'] = {'MSE': gb_mse, 'R2': gb_r2}

# Print all results
for model_name, metrics in model_results.items():
    print(f"\n{model_name} Results:")
    print(f"Test Mean Squared Error: {metrics['MSE']}")
    print(f"Test R-squared: {metrics['R2']}")
```

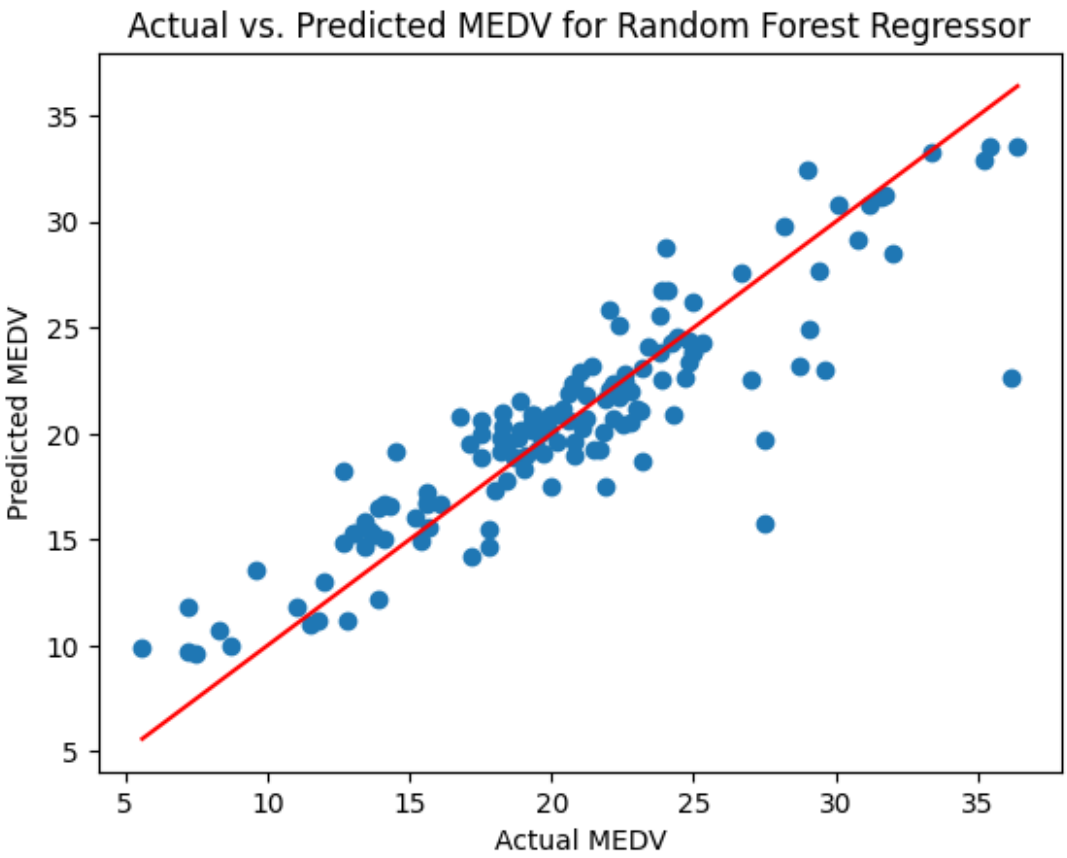
```
Decision Tree Results:
Test Mean Squared Error: 14.767500000000002
Test R-squared: 0.6067789497846272

Random Forest Results:
Test Mean Squared Error: 7.313212314285721
Test R-squared: 0.805267714462744

Gradient Boosting Results:
Test Mean Squared Error: 6.65162828449541
Test R-squared: 0.8228840183056332
```

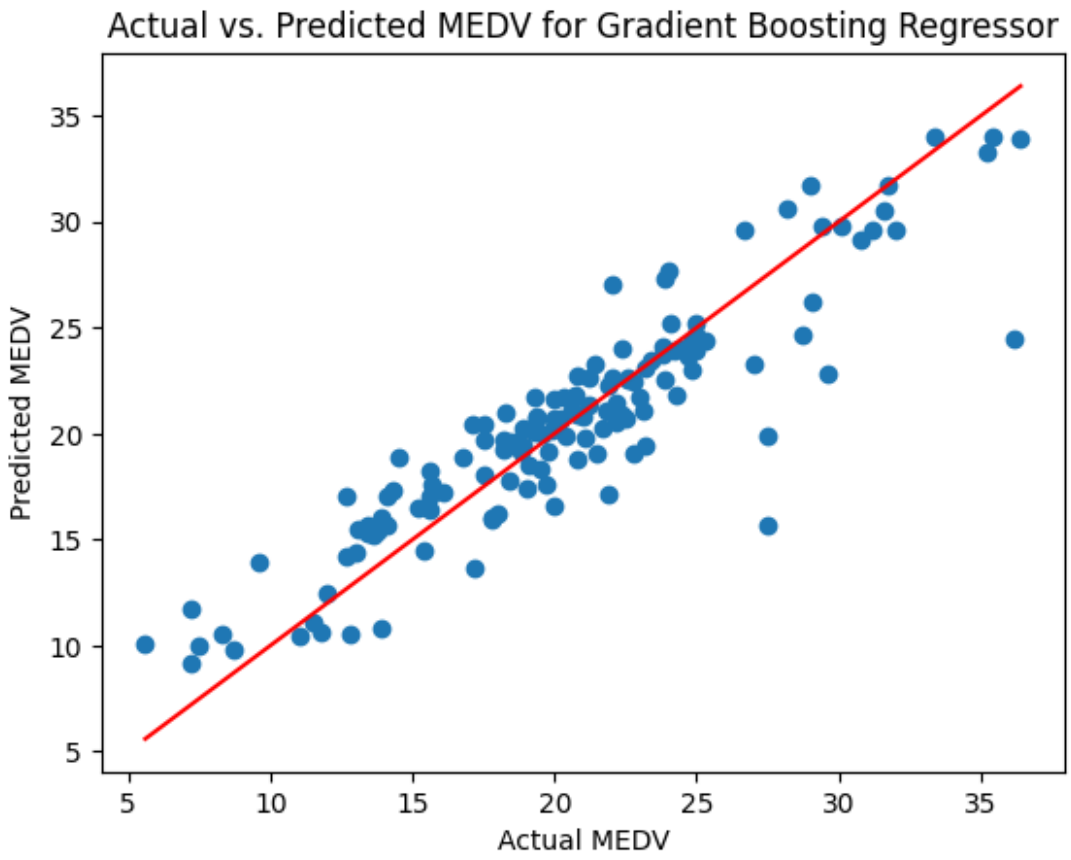
After removing outliers, still GB performs better.

```
plt.scatter(y_test, y_pred_dt)
plt.xlabel("Actual MEDV")
```

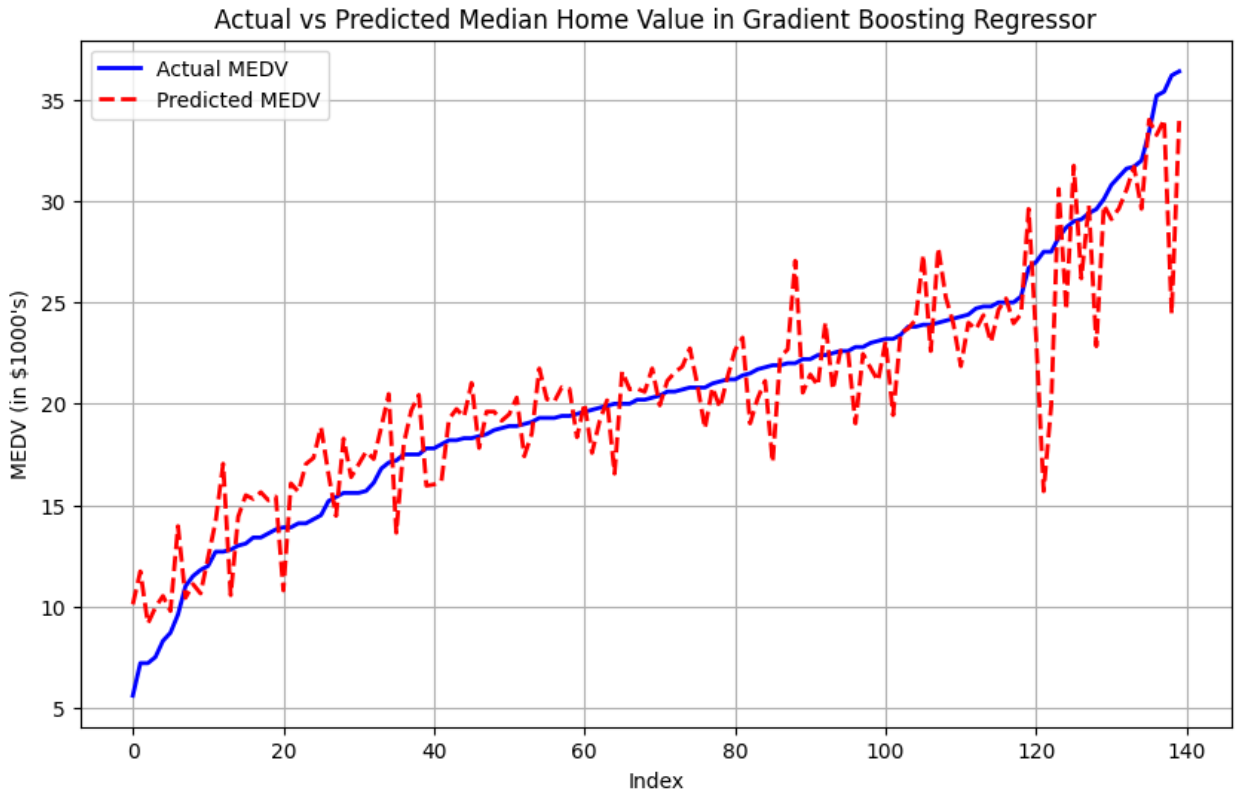
```
plt.ylabel("Predicted MEDV")
plt.title("Actual vs. Predicted MEDV for Decision Tree")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red')  # Line of best fit
plt.show()

plt.scatter(y_test, y_pred_rf)
plt.xlabel("Actual MEDV")
plt.ylabel("Predicted MEDV")
plt.title("Actual vs. Predicted MEDV for Random Forest Regressor")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red')  # Line of best fit
plt.show()

plt.scatter(y_test, y_pred_gb)
plt.xlabel("Actual MEDV")
plt.ylabel("Predicted MEDV")
plt.title("Actual vs. Predicted MEDV for Gradient Boosting Regressor")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red')  # Line of best fit
plt.show()
```




Actual vs. Predicted MEDV for Random Forest Regressor
Predicted MEDV
Actual MEDV
5
10
15
20
25
30
35




```
# Sort the test set for a cleaner line chart
sorted_index = np.argsort(y_test)
y_test_sorted = y_test.iloc[sorted_index]
y_pred_sorted = y_pred_gb[sorted_index]

# Plot actual vs predicted values as line charts
plt.figure(figsize=(10, 6))
plt.plot(y_test_sorted.values, label='Actual MEDV', color='b',
linewidth=2)
plt.plot(y_pred_sorted, label='Predicted MEDV', color='r',
linestyle='--', linewidth=2)

# Add labels, title, and legend
plt.xlabel("Index")
plt.ylabel("MEDV (in $1000's)")
plt.title("Actual vs Predicted Median Home Value in Gradient Boosting
Regressor")
plt.legend()
plt.grid(True)
plt.show()
```



Support Vector Regression (SVR)

Summary of Approaches with Support Vector Regression (SVR)

- Dataset split: 70% training, 30% testing (same used in previous models)
- Scaling: Applied standard scaling using StandardScaler.

1. **SVR with RBF Kernel (Simple Version):**

- Applied SVR with RBF kernal on all features wih feature scaling

- Results: **Test R-squared**: **0.6579564914830097**, Mean Squared Error **(MSE)**: **25.48672063849504**

1. **Optimized SVR with GridSearchCV**:

- **Hyperparameter Tuning**: Used **GridSearchCV** with **5-fold cross-validation** to tune **C**, **epsilon**, and **gamma** parameters for the RBF kernel in SVR. Where C (regularization) controls the balance between fitting the training data well, epsilon defines the margin within which no penalty is given to errors & gamma adjusts how much influence each data point has on the model, helping capture more complex patterns.

- Results: **Test R-squared**: **0.8387003182893529**, Mean Squared Error **(MSE)**: **6.057643781677839**

By applying hyperparameter tuning with GridSearchCV, the R-squared improved from **0.6579** to **0.8387**, and the MSE decreased from **25.48** to **6.05**, showing a significant performance gain and reduction in prediction errors.

Import Libraries

```
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV
import matplotlib.pyplot as plt
import seaborn as sns
```

Data Scaling

StandardScaler is used to normalize the data. SVR is sensitive to feature scaling, so this step ensures that all features are on the same scale

```
# Step 1: Data Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Hyperparameter Tuning with GridSearchCV

This section performs hyperparameter tuning using **GridSearchCV**, which systematically explores different combinations of model parameters such as **C** (regularization), **epsilon** (error margin), and **gamma** (kernel coefficient).

These hyperparameters are defined in param_grid and tested over multiple splits of the training data using 5-fold cross-validation. The goal is to find the best-performing combination based on minimizing negative mean squared error (MSE), ensuring the model generalizes well without overfitting.

```
# Step 2: Perform Hyperparameter Tuning using GridSearchCV
param_grid = {
    'C': [0.1, 1, 10, 100],   # Regularization parameter
    'epsilon': [0.01, 0.1, 0.5, 1],  # Epsilon in the epsilon-SVR
model
    'gamma': ['scale', 0.1, 0.01, 0.001]  # Kernel coefficient
}

# Step 3: Initialize SVR and use GridSearchCV
svr = SVR(kernel='rbf')
grid_search = GridSearchCV(svr, param_grid, cv=5,
scoring='neg_mean_squared_error')
grid_search.fit(X_train_scaled, y_train)

GridSearchCV(cv=5, estimator=SVR(),
             param_grid={'C': [0.1, 1, 10, 100], 'epsilon': [0.01,
```

```
0.1, 0.5, 1],
                          'gamma': ['scale', 0.1, 0.01, 0.001]},
              scoring='neg_mean_squared_error')
```

Getting the Best SVR Model & making predictions

Retrieves the best model (best_svr_model) after evaluating different combinations of hyperparameters from the GridSearchCV. After that it makes prediction of **MEDV** values

```
# Step 4: Get the best model after tuning
best_svr_model = grid_search.best_estimator_
y_pred_svr_best = best_svr_model.predict(X_test_scaled)
```

Performance Evaluation

Evaulates the model using two metrics:

1. **Mean Squared Error (MSE)**: Measures the average squared difference between actual and predicted values (lower is better).
2. **R-squared (R^2)**: Measures how well the model explains the variability in the target variable (closer to 1 is better).

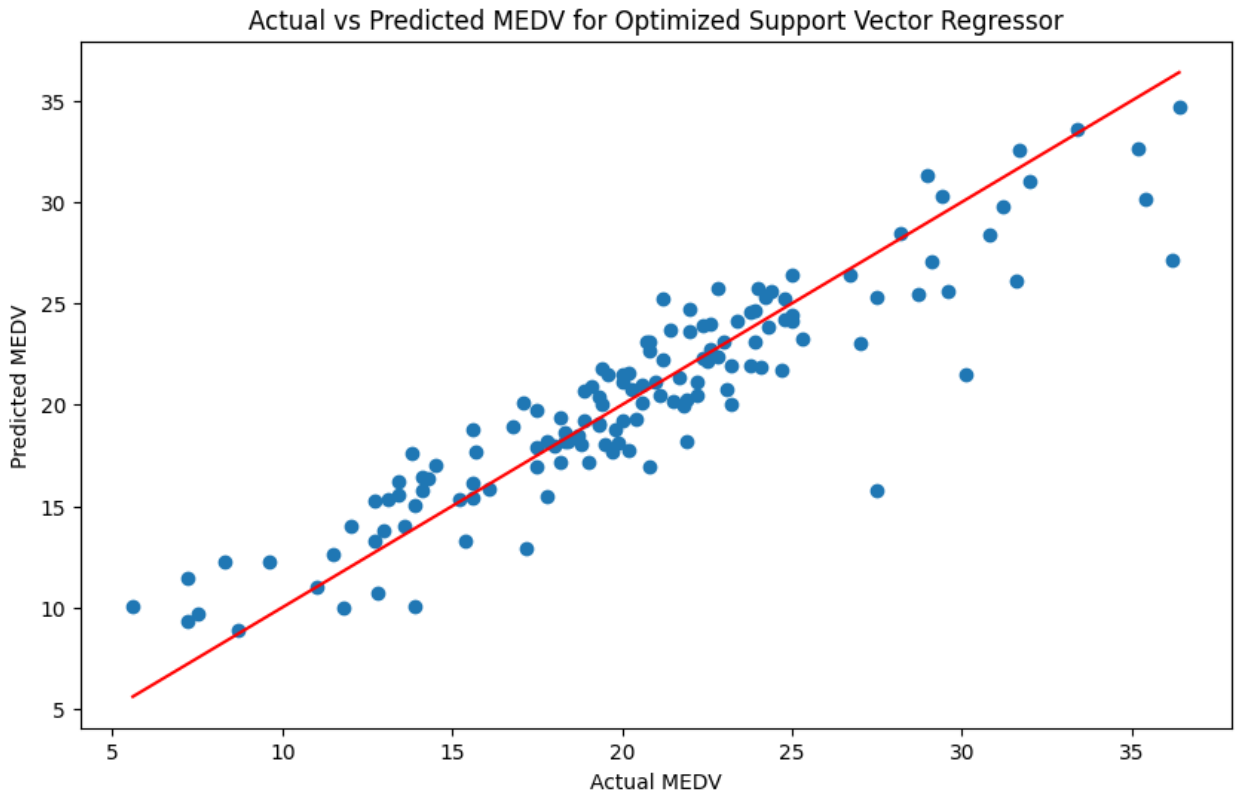
```
# Step 6: Calculate performance metrics
svr_mse_best = mean_squared_error(y_test, y_pred_svr_best)
svr_r2_best = r2_score(y_test, y_pred_svr_best)

print(f'SVR - Mean Squared Error: {svr_mse_best}')
print(f'SVR - R-squared: {svr_r2_best}')
```

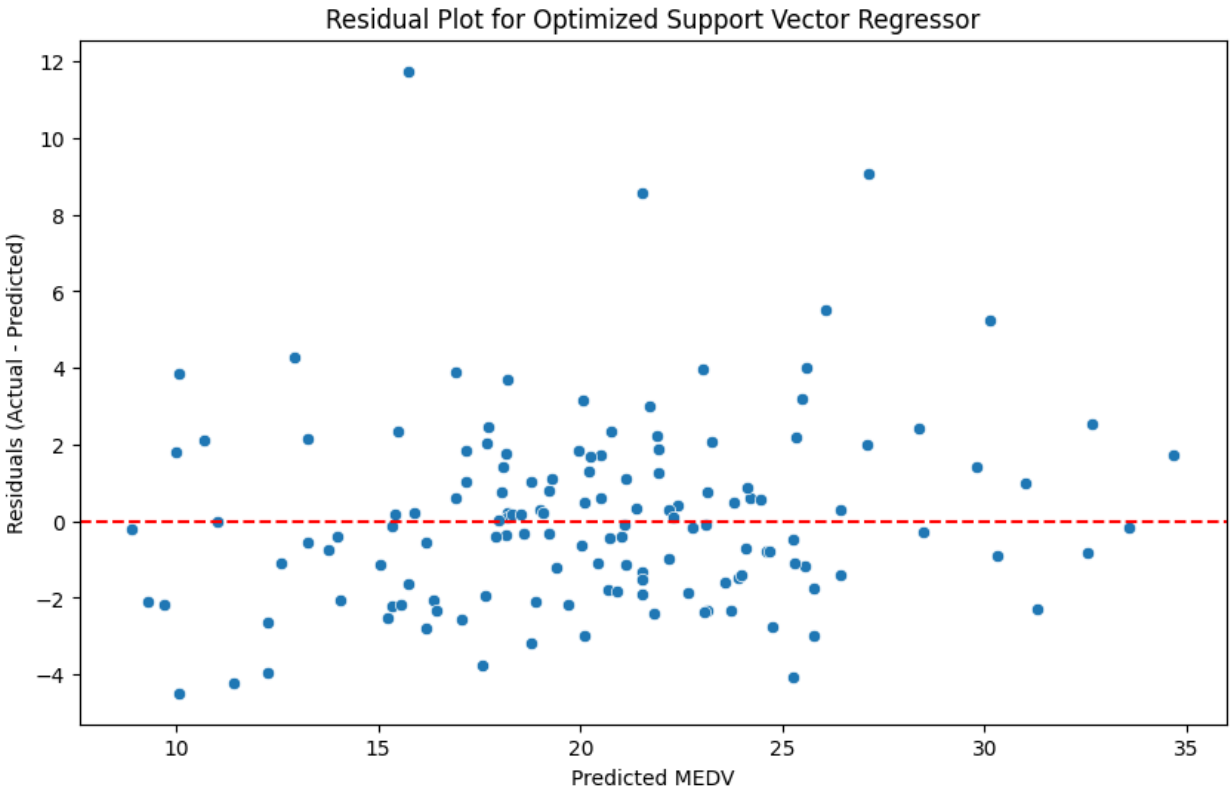
```
SVR - Mean Squared Error: 6.057643781677839
SVR - R-squared: 0.8387003182893529
```

Plotting Actual vs. Predicted Values of MEDV

```
# Step 7: Plot Actual vs Predicted MEDV
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_svr_best)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
color='red')
plt.xlabel('Actual MEDV')
plt.ylabel('Predicted MEDV')
plt.title('Actual vs Predicted MEDV for Optimized Support Vector
Regressor')
plt.show()


# Calculate residuals
residuals = y_test - y_pred_svr_best

# Plot residuals
```

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_pred_svr_best, y=residuals)
plt.axhline(0, color='red', linestyle='--')  # Horizontal line at 0
for reference
plt.xlabel('Predicted MEDV')
plt.ylabel('Residuals (Actual - Predicted)')
plt.title('Residual Plot for Optimized Support Vector Regressor')
plt.show()
```





Analytical Comparison of Models

Dataset Split & Scaling

- All models used a **70% training** and **30% testing** split.
- **Standard Scaling** was applied for the **SVR** model but not for other models.

Comparison Table of Results

- **Gradient Boosting Regressor** performed the **best**, with the highest **R-squared (0.8909)** and **lowest MSE (8.12)**, demonstrating its strength in capturing complex patterns.
- **Optimized SVR** showed significant improvement after tuning, achieving an **R-squared** of **0.8387** and **MSE** of **6.05**.
- **Random Forest** also performed well, with an **R-squared** of **0.8700** and **MSE** of **9.69**.
- **Linear Regression** had the **lowest R-squared (0.7166)**, indicating it explains less variability compared to the others, with an **MSE** of **21.11**.

Complete detailed results are shown below

Model	R-squared	MSE
Linear Regression	0.7166	21.11
SVR	0.6579	25.48
Optimized SVR	0.8387	6.05
Decision Tree	0.8486	11.28
Random Forest	0.8700	9.69
Gradient Boosting	0.8909	8.12

Analysis of Results

1. **Linear Regression**

- **Pros**: Simple, interpretable, and works well without scaling.
- **Cons**: R-squared is lower, meaning it explains less variability in the data. The model does not capture non-linear relationships well, leading to higher MSE compared to other models.
- **Best Use Case**: When interpretability is key, and the dataset is linear.

1. **Support Vector Regressor (SVR)**

- **Original SVR:** The **simple SVR (RBF Kernel**) without hyperparameter tuning had **lower R-squared (0.6579)** and higher MSE (25.48), indicating weaker performance.
- **Optimized SVR:** After applying **GridSearchCV** for hyperparameter tuning, the model showed a large improvement, achieving an **R-squared of 0.8387** and **MSE of 6.05**. It now competes closely with tree-based models, showing its strength in non-linear data after proper tuning.
- **Pros**: Flexible in capturing non-linear relationships. Performs better when optimized.
- **Cons**: Requires tuning to work effectively, can be slower to train compared to simpler models like linear regression.
- **Best Use Case**: For non-linear datasets, when capturing complex patterns is necessary, but requires tuning.

1. **Decision Tree**

- **Pros**: Simple to interpret, handles non-linear relationships, and faster to train
- **Cons**: Prone to overfitting, as shown by its lower R-squared compared to ensemble methods like Random Forest or Gradient Boosting.
- **Best Use Case**: When simplicity is needed, but overfitting can be controlled.

1. **Random Forest**

- **Pros**: Higher R-squared (0.8700) and lower MSE (9.69), showing it captures more data variability. It reduces overfitting by averaging multiple decision trees.
- **Cons**: Slower to train and interpret compared to simpler models.
- **Best Use Case**: When accuracy is key, and interpretability is less of a priority.

1. **Gradient Boosting Regressor**

- **Pros**: Highest R-squared (0.8909) and lowest MSE (8.12), indicating the best performance among all models. It excels at correcting errors in sequential training steps
- **Cons**: Computationally expensive and sensitive to hyperparameters.
- **Best Use Case**: When high accuracy is critical, especially for non-linear and complex datasets.

Conclusion

Based on the dataset's specific patterns, such as the presence of **non-linear relationships** between features like **LSTAT** and **RM** (which had high importance in tree-based models) and the **target variable (MEDV)**, **Gradient Boosting Regressor** excels by capturing these complexities. It consistently outperformed other models by handling **non-linearity** and reducing errors effectively. While **Optimized SVR** also improved performance, Gradient Boosting's ability to sequentially correct errors and handle important features like **DIS** and **PTRATIO** makes it the most suitable model for this dataset.